



UT 1. Introducción a la creación de interfaces con editores visuales

Desarrollo de interfaces

Desarrollo de aplicaciones multiplataforma

Curso académico 2025/2026 - Edición Online - Rev.2025070801

Índice

Ficha de la Unidad de Trabajo	3
Índice de la unidad de trabajo	3
1. Introducción a los entornos de desarrollo visual	4
1.2. Características esenciales de los entornos visuales	5
2. Creación de interfaces gráficas básicas	7
3. Gestión de propiedades de componentes	9
3.1. Propiedades generales: texto, tamaño, color, nombre, visibilidad	10
3.2. Uso del panel de propiedades del editor	11
3.3. Cambio de propiedades en tiempo de diseño	12
Propiedades y accesibilidad	12
4. Análisis y modificación del código generado	13
¿Qué has aprendido?	16
Fuentes de información	17
Glosario	18

Ficha de la Unidad de Trabajo

Nombre de la UT: Introducción a la creación de interfaces con editores visuales

Carga horaria: 8 horas

RAs trabajados: RA1

CEs trabajados: RA1CEa, RA1CEe

Índice de la unidad de trabajo

UT 1. Introducción a la creación de interfaces con editores visuales

1. Introducción a los entornos de desarrollo visual

- 1.1. ¿Qué es un editor visual?
- 1.2. Características esenciales de los entornos visuales
- 1.3. Comparativa con el diseño mediante código
- 1.4. Ventajas del diseño asistido

2. Creación de interfaces gráficas básicas

- 2.1. Uso de asistentes para generar formularios
- 2.2. Componentes de interfaz: botones, etiquetas, cuadros de texto

3. Gestión de propiedades de componentes

- 3.1. Propiedades generales: texto, tamaño, color, nombre, visibilidad
- 3.2. Uso del panel de propiedades del editor
- 3.3. Cambio de propiedades en tiempo de diseño

4. Análisis y modificación del código generado

- 4.1. Relación entre el diseño visual y el código fuente
- 4.2. Identificación de clases, métodos y propiedades autogeneradas
- 4.3. Modificación de comportamientos básicos desde el código

1. Introducción a los entornos de desarrollo visual

En la actualidad, el diseño de interfaces gráficas de usuario (GUI, por sus siglas en inglés) es un aspecto fundamental en el desarrollo de aplicaciones. Los usuarios esperan interactuar con programas intuitivos, visuales y fáciles de usar. Para facilitar este proceso, los entornos de desarrollo integrados (IDE) incluyen herramientas visuales que permiten construir interfaces de forma asistida. Estas herramientas son conocidas como **editores visuales**.

El uso de editores visuales supone una evolución con respecto a los métodos tradicionales de diseño basados en la escritura manual de código. Gracias a ellos, los desarrolladores pueden centrarse en la funcionalidad, al tiempo que diseñan interfaces claras y organizadas sin preocuparse por los aspectos técnicos más detallados del lenguaje de programación subyacente.

1.1. ¿Qué es un editor visual?

Un **editor visual** es una herramienta software que permite construir la interfaz gráfica de una aplicación mediante operaciones visuales como arrastrar y soltar elementos (componentes) sobre una superficie de diseño, sin necesidad de escribir directamente el código que define esos elementos.

En otras palabras, actúa como un "**constructor gráfico de pantallas**", donde el desarrollador puede ver de forma inmediata cómo se va formando la interfaz de usuario mientras trabaja con ella. A medida que se insertan elementos como botones, etiquetas o cuadros de texto, el editor va generando automáticamente el código necesario en segundo plano.

Algunos ejemplos de editores visuales ampliamente utilizados en entornos reales son:

- **Java Swing con NetBeans**
- **Windows Forms o WPF en Visual Studio**
- **Android Studio con XML y diseño visual**
- **Qt Designer (para C++ o Python con PyQt)**

El uso de editores visuales es común en aplicaciones de escritorio, móviles y web, ya que facilita el diseño centrado en el usuario y permite un prototipado rápido.

1.2. Características esenciales de los entornos visuales

Los entornos visuales comparten una serie de funcionalidades clave que los hacen especialmente útiles en el diseño de interfaces gráficas. A continuación se describen las más importantes:

- **Diseño mediante arrastrar y soltar:** Los controles visuales (botones, campos de texto, menús, etc.) se arrastran desde un panel de herramientas y se sueltan sobre una superficie de diseño, lo que permite componer visualmente la pantalla.
- **Panel de propiedades:** Permite modificar rápidamente los atributos de cada componente, como su texto, tamaño, color, tipo de letra, visibilidad, comportamiento, etc.
- **Vista previa en tiempo real:** Se puede observar cómo quedará la interfaz final sin necesidad de compilar ni ejecutar la aplicación.
- **Asociación de eventos:** Los editores permiten acceder fácilmente a los eventos de cada componente (como un clic en un botón) y vincularlos con acciones programadas en el código.
- **Sincronización con el código fuente:** A medida que se diseña visualmente, se genera el código fuente automáticamente, que puede ser revisado y modificado por el desarrollador en cualquier momento.
- **Plantillas y asistentes:** Muchos editores ofrecen plantillas predefinidas y asistentes para la creación de formularios comunes (como ventanas de inicio de sesión, formularios de contacto, etc.).

Estas características hacen que el proceso de diseño sea más eficiente, especialmente en etapas iniciales del desarrollo o cuando se requiere validar rápidamente un prototipo con un cliente.

1.3. Comparativa con el diseño mediante código

Antes de la aparición de editores visuales, todas las interfaces debían ser definidas mediante código, especificando cada componente, su posición, tamaño y comportamiento manualmente. Este enfoque sigue siendo válido y en ocasiones necesario, pero presenta algunas desventajas frente al diseño visual.

Diseño visual	Diseño por código
Intuitivo y rápido	Requiere conocimiento detallado del lenguaje
Permite ver la interfaz en tiempo real	Solo se visualiza al compilar y ejecutar
Ideal para prototipado	Más lento para cambios frecuentes
Genera código automáticamente	El desarrollador escribe cada línea a mano
Menor control sobre el código generado	Mayor precisión y personalización

Ventaja del código manual: permite personalizaciones avanzadas, acceso directo a las propiedades del framework y optimización fina del comportamiento de la interfaz.

Ventaja del diseño visual: permite desarrollar rápidamente prototipos funcionales sin necesidad de conocimientos profundos de programación.

Lo más habitual en el desarrollo profesional es **complementar ambos enfoques**, comenzando con un diseño visual que luego se ajusta o personaliza desde el código cuando sea necesario.

1.4. Ventajas del diseño asistido

El uso de editores visuales ofrece una serie de beneficios que impactan directamente en la productividad y calidad del desarrollo de interfaces. Estas son sus principales ventajas:

- **Rapidez en la creación de prototipos:** Ideal para presentar maquetas funcionales a usuarios o clientes y obtener retroalimentación temprana.
- **Facilidad de uso:** El aprendizaje inicial es accesible, incluso para perfiles con poca experiencia en programación.
- **Reducción de errores de sintaxis:** Al generar automáticamente el código asociado a la interfaz, se evitan errores comunes como cierres de etiquetas, errores tipográficos o referencias a componentes inexistentes.

- **Ahorro de tiempo en tareas repetitivas:** Los editores visuales permiten reutilizar formularios, duplicar pantallas o clonar componentes sin tener que repetir manualmente el código.
- **Mejora del trabajo en equipo:** Diseñadores y programadores pueden colaborar más fácilmente, ya que el diseño visual se entiende sin necesidad de leer el código.
- **Mayor accesibilidad al diseño:** Profesionales de otras disciplinas (como diseñadores gráficos, testers o analistas) pueden participar en la creación o revisión de interfaces sin tener conocimientos técnicos profundos.
- **Vista previa inmediata:** Al ver en tiempo real cómo va quedando la interfaz, se pueden corregir y ajustar errores de distribución, tamaño o estética de forma inmediata.

Estas ventajas convierten al diseño asistido en una herramienta esencial dentro del proceso de desarrollo de aplicaciones modernas, en especial cuando la experiencia del usuario es un factor determinante del éxito del producto.

2. Creación de interfaces gráficas básicas

La creación de interfaces gráficas básicas es el primer paso para comprender cómo se diseñan las aplicaciones modernas. En esta sección se cubrirá el uso de asistentes para generar formularios básicos y se explicarán los componentes fundamentales que componen una interfaz.

2.1. Uso de asistentes para generar formularios

Los **asistentes de formularios** son herramientas que permiten a los usuarios generar formularios con varios controles de manera rápida y sencilla. Un asistente de formularios guía al desarrollador en el proceso de creación, proporcionándole una serie de pasos a seguir para seleccionar los componentes necesarios y definir sus propiedades.

Pasos para crear un formulario básico:

1. **Seleccionar los componentes:** Utilizando el asistente, el desarrollador puede elegir los controles que necesita para su formulario, como **botones**, **etiquetas** y **cuadros de texto**.
2. **Organizar los componentes:** El asistente permite al desarrollador organizar los controles en la interfaz mediante una vista previa, posicionando los elementos donde se necesiten.
3. **Configurar las propiedades de los controles:** Los asistentes también permiten modificar las propiedades de los controles, como el tamaño, el color y el texto de los botones, etiquetas y cuadros de texto.

2.2. Componentes de interfaz: botones, etiquetas, cuadros de texto

Los componentes son los bloques básicos de cualquier interfaz gráfica. Los más comunes son los **botones**, **etiquetas** y **cuadros de texto**.

- **Botones:** Los botones permiten a los usuarios interactuar con la aplicación mediante un clic. Pueden ser utilizados para realizar diversas acciones, como enviar información o cerrar una ventana.
 - Propiedades importantes:
 - **Texto:** Lo que aparece en el botón.
 - **Color:** De fondo y de texto.
 - **Tamaño:** Para ajustar la apariencia.
 - **Ejemplo:** Crear un botón de "Enviar" en un formulario de registro.
- **Etiquetas:** Son elementos que permiten mostrar información estática en la interfaz, como mensajes o títulos. Las etiquetas no permiten la interacción del usuario.

- Propiedades importantes:
 - **Texto:** Lo que muestra la etiqueta.
 - **Color de fondo:** Para mejorar la visibilidad.
 - **Tamaño de fuente:** Para hacerla más legible.
- **Ejemplo:** Crear una etiqueta que indique "Por favor ingrese su correo electrónico" en un formulario.
- **Cuadros de texto:** Permiten a los usuarios ingresar información en la aplicación. Se usan comúnmente para que los usuarios escriban su nombre, dirección de correo electrónico, etc:
 - Propiedades importantes:
 - **Texto:** Lo que aparece en el cuadro de texto.
 - **Tamaño:** Para permitir la entrada de información más extensa.
 - **Formato:** Para definir si el campo debe aceptar solo letras, números o contraseñas.
 - **Ejemplo:** Crear un cuadro de texto para ingresar el correo electrónico del usuario.

3. Gestión de propiedades de componentes

Una vez insertados los componentes básicos en un formulario, es necesario **personalizarlos para que se ajusten al diseño visual, la funcionalidad y la lógica de la aplicación**. Esta personalización se realiza mediante la modificación de sus **propiedades**, que pueden ajustarse tanto en tiempo de diseño (desde el editor visual) como directamente desde el código fuente.

Dado que a lo largo de esta unidad trabajaremos con ejemplos apoyados en **HTML, CSS y JavaScript**, nos centraremos en cómo se configuran estos elementos en entornos web. Este enfoque nos permitirá comprender con claridad cómo se definen

propiedades como el contenido textual, el estilo visual (colores, tamaños, tipografía) o el comportamiento de los elementos interactivos mediante eventos, partiendo de tecnologías ampliamente conocidas y utilizadas en el desarrollo de interfaces gráficas.

El conocimiento de las propiedades más habituales y de cómo afectan al comportamiento y la apariencia del componente es esencial para construir interfaces claras, accesibles y funcionales.

3.1. Propiedades generales: texto, tamaño, color, nombre, visibilidad

Cada componente visual tiene un conjunto de **propiedades** que definen su aspecto, comportamiento y relación con el entorno gráfico.

Propiedades más comunes:

- **Text (texto)**

Define el contenido textual que se muestra en un botón, etiqueta o cualquier componente que presente información al usuario.

Ejemplo: Un botón con la propiedad Text = "Guardar" mostrará esa palabra como etiqueta visible.

- **Name (nombre interno)**

Es el nombre identificador del componente dentro del código. No es visible para el usuario, pero permite referenciar ese elemento desde la programación.

Ejemplo: Un cuadro de texto llamado txtUsuario puede usarse en el código para obtener su contenido.

- **Size (tamaño)**

Especifica el ancho y alto del componente. Se puede ajustar visualmente desde los bordes del control o estableciendo valores numéricos.

Ejemplo: Un botón puede tener Width = 100 px y Height = 30 px.

- **Location (posición)**

Define la ubicación del componente dentro del formulario, normalmente en coordenadas (x, y).

- **BackColor y ForeColor (color de fondo y de texto)**

Personalizan los colores del componente, afectando tanto a su apariencia como a su legibilidad.

- **Font (tipografía)**

Permite establecer tipo de letra, tamaño, negrita, cursiva, etc.

- **Enabled (habilitado)**

Indica si el componente está activo o desactivado. Un componente deshabilitado se ve atenuado y no responde a la interacción del usuario.

- **Visible (visible)**

Controla si el componente es visible o está oculto en la interfaz.

Importancia de usar nombres descriptivos:

Aunque los editores visuales asignan nombres automáticos (como `button1`, `label3`), es buena práctica cambiar estos nombres por otros descriptivos (`btnGuardar`, `lblMensaje`, `txtCorreo`). Esto facilita la lectura y mantenimiento del código fuente.

3.2. Uso del panel de propiedades del editor

La mayoría de entornos visuales incluye un **panel de propiedades** que muestra todas las configuraciones posibles del componente seleccionado. Este panel permite modificar fácilmente los atributos sin necesidad de escribir código.

Funciones del panel de propiedades:

- Ver en una sola vista todas las propiedades disponibles.
- Modificar textos, colores, tamaños, etc., mediante cuadros de texto o menús desplegables.
- Cambiar el nombre lógico del componente.
- Establecer restricciones (por ejemplo, limitar el número máximo de caracteres en un campo de texto).
- Agrupar propiedades por categorías: apariencia, comportamiento, diseño, etc.

Ejemplo práctico: Al seleccionar un `TextBox` en Visual Studio, el panel de propiedades permite establecer `Text = ""`, `MaxLength = 50`, `PasswordChar = *`, entre otros.

Esta funcionalidad acelera el desarrollo y evita errores de escritura en código. Además, permite ver el efecto de los cambios de forma inmediata en la vista de diseño.

3.3. Cambio de propiedades en tiempo de diseño

Modificar propiedades en **tiempo de diseño** significa hacerlo directamente desde el entorno visual antes de ejecutar la aplicación. Esto es diferente al cambio de propiedades en **tiempo de ejecución**, que se realiza desde el código fuente mientras la aplicación está activa.

Ventajas del tiempo de diseño:

- Visualización inmediata de los cambios.
- Permite alinear y dimensionar los controles con precisión.
- Simplifica el ajuste inicial de estilos y disposición.

Buenas prácticas en tiempo de diseño:

- Usar alineaciones automáticas y guías del editor para distribuir componentes de forma ordenada.
- Probar diferentes combinaciones de colores y fuentes para verificar la legibilidad.
- Verificar que todos los componentes tengan nombres únicos y descriptivos.

Consejo útil: Los editores visuales permiten seleccionar múltiples componentes a la vez para cambiar propiedades comunes (como fuente o tamaño) de forma simultánea.

Propiedades y accesibilidad

Algunas propiedades también afectan la **accesibilidad** de la interfaz. Por ejemplo:

- El contraste entre el color del texto y el fondo debe ser suficiente.
- Los textos deben ser legibles y suficientemente grandes.
- Los campos deben estar correctamente etiquetados para lectores de pantalla.

Estas consideraciones son fundamentales para garantizar que la interfaz sea usable por todos los usuarios, incluyendo aquellos con alguna discapacidad visual o motriz.

4. Análisis y modificación del código generado

El diseño visual de interfaces no se limita a la disposición gráfica de elementos en pantalla. Cada vez que insertamos un botón, una etiqueta o un campo de texto en un formulario mediante un editor visual, el entorno está generando simultáneamente una serie de instrucciones en código fuente que dan forma y comportamiento a esos elementos.

Comprender la relación entre el diseño visual y el código asociado es esencial para poder adaptar, ampliar o depurar la funcionalidad de una interfaz. En este bloque se aprenderá a identificar esa correspondencia, reconocer las estructuras básicas del código generado y entender cómo pueden personalizarse ciertos comportamientos desde la programación.

4.1. Relación entre el diseño visual y el código fuente

Cada acción que realizamos en el editor visual —como colocar un componente, cambiar su tamaño, modificar un texto o definir una propiedad— genera automáticamente líneas de código en segundo plano.

Esta traducción ocurre en tres niveles principales:

- Declaración de los componentes: el sistema crea una representación en código de cada elemento visual que insertamos. Por ejemplo, si añadimos un botón, se genera una instancia correspondiente en el lenguaje del proyecto.
- Configuración de sus propiedades: las propiedades que establecemos gráficamente (como el texto visible, el tamaño o el color) se reflejan en el código mediante asignaciones o llamadas a métodos.
- Asociación de eventos: si vinculamos una acción a un componente (por ejemplo, al pulsar un botón), el editor prepara también la estructura necesaria para que podamos definir su comportamiento en código.

Esta relación es automática y bidireccional: los cambios que hacemos en la interfaz se reflejan en el código, y en muchos casos, las modificaciones que realizamos directamente en el código también afectan a la interfaz, si el editor lo permite.

Ejemplo conceptual: cuando colocamos un botón llamado "Aceptar" en una ventana y escribimos su texto visible, el entorno registra internamente ese nombre, lo declara como objeto del tipo adecuado y le asigna esa propiedad de texto, que luego puede ser leída, modificada o utilizada desde la lógica del programa.

4.2. Identificación de clases, métodos y propiedades autogeneradas

El código que acompaña a una interfaz gráfica no lo escribe el desarrollador manualmente, sino que lo genera automáticamente el editor a partir de lo que se construye visualmente. Aun así, es importante poder leerlo, interpretarlo y modificarlo con criterio.

Algunas características clave del código generado son:

- Se estructura en clases y métodos organizados: cada formulario o ventana suele tener su propia clase, donde se definen todos los componentes que contiene.
- Contiene declaraciones de variables o atributos: cada componente se guarda en una variable que lo representa dentro del código.
- Incluye secciones de inicialización: en estas se asignan valores a las propiedades y se establecen las relaciones entre componentes y su disposición en pantalla.
- Presenta puntos de conexión con la lógica del programa: por ejemplo, si se desea que al hacer clic en un botón ocurra algo, el editor genera una estructura vacía o plantilla que el desarrollador puede completar.

En función del lenguaje o entorno utilizado, estos elementos se agrupan en archivos diferentes, métodos especiales o bloques de código ocultos para proteger el diseño. Sin embargo, la lógica general siempre es la misma: cada objeto gráfico tiene un reflejo directo en el código fuente que lo gestiona.

4.3. Modificación de comportamientos básicos desde el código

Aunque la mayoría de los ajustes visuales pueden hacerse desde el editor gráfico, hay situaciones en las que es necesario o conveniente realizar cambios directamente en el código. Esto suele ocurrir cuando:

- Se necesita modificar una propiedad durante la ejecución del programa (por ejemplo, cambiar el texto de un botón cuando se complete una acción).
- Se quiere añadir una validación o reacción específica cuando el usuario interactúa con un componente.
- Se requiere modificar dinámicamente el aspecto o disposición de la interfaz según condiciones particulares.

Acciones habituales desde código:

- Cambiar el contenido de un texto visible.
- Habilitar o deshabilitar componentes según estados.
- Ocultar o mostrar elementos de forma condicional.
- Capturar lo que el usuario escribe y utilizarlo en cálculos, búsquedas o verificaciones.
- Conectar componentes con otras partes del sistema (por ejemplo, enviar datos a una base de datos).

Estas acciones requieren que el desarrollador identifique los componentes por su nombre lógico y utilice las instrucciones adecuadas del lenguaje de programación del entorno para manipularlos.

¿Qué has aprendido?

- **Identificación y uso de editores visuales** como herramientas para diseñar interfaces gráficas sin necesidad de programar desde cero.
- **Diferencias entre diseño visual y diseño por código**, entendiendo cuándo conviene usar cada uno.
- **Creación de formularios** mediante asistentes guiados, agilizando el proceso de maquetación de pantallas.
- **Inserción y configuración de componentes básicos**: botones, etiquetas y cuadros de texto.
- **Gestión de propiedades** desde el editor visual, ajustando atributos como el texto, el color, el tamaño, el nombre y la visibilidad de los controles.
- **Relación entre el diseño visual y el código generado automáticamente**, comprendiendo su estructura y aprendiendo a modificarlo para añadir lógica personalizada.

Reflexión final

Esta primera unidad me ha permitido entender que el diseño de una aplicación no empieza directamente en el código, sino en **la construcción visual de la interfaz**, orientada al usuario. Ahora sé que los editores visuales no son solo una herramienta para trabajar más rápido, sino también un puente entre la experiencia de usuario y el funcionamiento interno de una aplicación. He aprendido a trabajar de forma más eficiente, pero también más consciente: cada componente que coloco en una interfaz tiene un propósito, una propiedad y un comportamiento que puedo controlar.

Comprender cómo se genera y organiza el código a partir del diseño gráfico me abre las puertas para personalizar interfaces, depurar errores y dar un paso más allá: construir aplicaciones pensadas no solo para funcionar, sino para ser utilizadas de forma intuitiva, profesional y accesible.

Fuentes de información

Deitel, P. J., & Deitel, H. M. (2012). *Java: Cómo programar* (9.^a ed.). Pearson Educación.

Microsoft. (s.f.). *Documentación de Windows Forms*. <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/>

Oracle. (s.f.). *Guía de introducción a Java Swing*.
<https://docs.oracle.com/javase/tutorial/uiswing/>

Oracle. (s.f.). *Diseño de GUI con NetBeans IDE*.
<https://netbeans.apache.org/kb/docs/java/quickstart-gui.html>

Google. (s.f.). *Desarrolladores de Android – Diseñar interfaces con XML*.
<https://developer.android.com/guide/topics/ui/declaring-layout>

Qt Company. (s.f.). *Qt Designer Manual*. <https://doc.qt.io/qt-6/qtdesigner-manual.html>

Nielsen, J. (1993). *Usability Engineering*. Morgan Kaufmann.

Glosario

Editor visual: Un editor visual es una herramienta que permite crear interfaces gráficas mediante una interfaz gráfica, utilizando componentes predefinidos sin necesidad de escribir código manualmente.

Componente: Es un elemento básico en una interfaz gráfica, como un botón, cuadro de texto o etiqueta, que permite la interacción del usuario o la visualización de datos.

Arrastrar y soltar (Drag and Drop): Es una acción que permite mover componentes de una interfaz gráfica desde un área de trabajo a otro, simplemente arrastrándolos con el ratón.

Propiedad: Es un atributo de un componente que puede ser modificado para cambiar su apariencia o comportamiento, como el tamaño, color o texto de un botón.

Formulario: Es una interfaz gráfica que permite al usuario introducir y enviar datos, comúnmente compuesta por varios componentes como cuadros de texto, botones y etiquetas.

Asistente de formulario: Herramienta que guía al desarrollador en el proceso de creación de un formulario, sugiriendo y configurando automáticamente los componentes necesarios.

Evento: Es una acción que ocurre como resultado de una interacción del usuario con la interfaz, como hacer clic en un botón o escribir en un cuadro de texto.

Código fuente: Es el conjunto de instrucciones escritas en un lenguaje de programación que define el comportamiento de los componentes y la lógica de la aplicación.

Vista previa: Es una visualización en tiempo real del diseño de la interfaz, que permite ver cómo quedará la aplicación antes de realizar cambios en el código o ejecutarla.

Interfaz gráfica de usuario (GUI): Es un tipo de interfaz que permite a los usuarios interactuar con la aplicación mediante gráficos, imágenes, botones y otros elementos visuales en lugar de utilizar solo texto.

Actividades

Actividad 1: Exploración de un Editor Visual

Descripción:

Se deberá abrir un editor visual (por ejemplo, NetBeans, Visual Studio o Eclipse), identificar sus componentes principales y crear una interfaz básica mediante HTML y CSS con botones, etiquetas y cuadros de texto.

RA asociados: RA1CEa, RA1CEe

Rúbrica de Evaluación

CE	Excelente (9-10)	Notable (7-8)	Aprobado (5-6)	Suspenso (<5)
RA1CEa	Utiliza el editor visual con soltura, crea una interfaz funcional, coherente y bien organizada.	Utiliza el editor visual correctamente, crea una interfaz funcional aunque básica.	Utiliza el editor con dificultades, la interfaz está incompleta o poco clara.	No utiliza correctamente el editor o la interfaz no es funcional.
RA1CEe	Identifica claramente las ventajas del editor visual y las explica con ejemplos concretos y pertinentes.	Reconoce las ventajas principales del editor visual y las describe de forma general.	Muestra comprensión parcial de las ventajas, con explicaciones superficiales.	No identifica ventajas del editor visual o muestra confusión conceptual.

Actividad 2: Creación de Formularios con Asistentes

Descripción:

Se deberán utilizar los asistentes del editor visual para crear un formulario con varios campos de entrada, botones y etiquetas, sin escribir código manual. Se puede utilizar algún plugin de VSCode para que te autogener el código.

RA asociado: RA1CEa

Rúbrica de Evaluación

Indicador	Excelente (9-10)	Notable (7-8)	Aprobado (5-6)	Suspenso (<5)
Uso de asistentes	Crea un formulario completo y funcional usando los asistentes correctamente.	Crea un formulario básico con uso adecuado de asistentes.	Crea un formulario con errores importantes o incompleto.	No utiliza asistentes o el formulario no funciona correctamente.

Actividad 3: Modificación de Propiedades y Diseño en Tiempo Real

Descripción:

Se deberán modificar propiedades de componentes (texto, color, tamaño, etc.) y reorganizar la interfaz en tiempo de diseño, sin escribir código. En VSCode, por ejemplo, puedes cambiar el color de una etiqueta CSS solo con un clic del ratón.

RA asociado: RA1CEa

Rúbrica de Evaluación

Indicador	Excelente (9-10)	Notable (7-8)	Aprobado (5-6)	Suspenso (<5)
Gestión de propiedades	Modifica propiedades con precisión y mejora la usabilidad de la interfaz de forma evidente.	Modifica propiedades correctamente con algunos ajustes que mejoran la funcionalidad.	Modifica propiedades básicas, pero con errores menores o sin impacto claro.	No modifica propiedades o lo hace de forma incorrecta o confusa.

Actividad 4: Análisis del Código Generado por el Editor Visual

Descripción:

Debemos crear una interfaz gráfica sencilla con el editor visual (por ejemplo, un formulario con botones y campos de texto en HTML), y luego observar el código generado automáticamente. A continuación, se debe identificar qué parte del diseño corresponde a cada fragmento de código y explicar brevemente esa relación

RA asociado: RA1CEe

Rúbrica de Evaluación

Indicador	Excelente (9-10)	Notable (7-8)	Aprobado (5-6)	Suspenso (<5)
Análisis del código	Comprende completamente el código generado por el editor visual y realiza modificaciones coherentes y eficaces.	Comprende el código con algunas dudas, pero realiza cambios funcionales adecuados.	Comprende parcialmente el código, con capacidad limitada para modificarlo.	No comprende el código generado o realiza cambios erróneos o incoherentes.

Tabla de relación entre contenidos y criterios de evaluación (CE)

Contenidos	Criterios de Evaluación (CE)
Introducción a los entornos de desarrollo visual	RA1CEa: Comprende y utiliza un editor visual para la creación de interfaces gráficas.
Características generales de un editor visual	RA1CEe: Identifica y comprende las ventajas de utilizar editores visuales.
Ventajas del diseño asistido	RA1CEe: Valora las ventajas y facilita el uso de herramientas de diseño asistido.
Creación de interfaces gráficas básicas	RA1CEa: Utiliza asistentes para generar interfaces básicas mediante un editor visual.
Uso de asistentes para generar formularios	RA1CEa: Crea interfaces utilizando los asistentes disponibles en el editor visual.
Componentes de interfaz: botones, etiquetas, cuadros de texto	RA1CEa: Selecciona y organiza componentes básicos de interfaz (botones, cuadros de texto, etiquetas).
Gestión de propiedades de componentes	RA1CEa: Modifica las propiedades de los controles dentro del editor visual.
Modificación en tiempo de diseño	RA1CEa: Realiza ajustes y modificaciones de los controles en tiempo real, sin necesidad de escribir código.
Análisis y modificación del código generado	RA1CEe: Comprende y edita el código generado por el editor visual para personalizar la interfaz.
Relación entre diseño visual y código subyacente	RA1CEe: Entiende la relación entre la estructura visual y el código generado, aplicando cambios de manera coherente.

Rúbrica de evaluación

Indicador de logro	Excelente (10-9)	Notable (8-7)	Aprobado (6-5)	Insuficiente (<5)
Cumplimiento de los objetivos de la UT	Se han alcanzado todos los objetivos de la unidad, creando una interfaz gráfica completa, funcional, intuitiva y estéticamente atractiva.	Se han alcanzado la mayoría de los objetivos de la unidad, con una interfaz funcional y bien diseñada, aunque algunos aspectos podrían mejorarse.	Se ha cumplido con los objetivos básicos de la unidad, pero su interfaz presenta deficiencias importantes en funcionalidad o diseño.	Se han alcanzado los objetivos fundamentales de la unidad, pero, la interfaz creada presenta graves fallos en funcionalidad, diseño o ambos.
Aplicación de conocimientos sobre editores visuales	Se ha demostrado un dominio completo de los editores visuales, utilizando de forma avanzada todas sus herramientas y componentes.	Se ha demostrado un buen manejo de los editores visuales, utilizando correctamente las herramientas y componentes, aunque con algunas limitaciones.	Se ha mostrado un conocimiento básico de los editores visuales y emplea algunas herramientas de forma limitada o incorrecta.	Se han tenido dificultades para utilizar el editor visual y no emplea adecuadamente sus herramientas o componentes.
Creatividad y diseño visual	La interfaz es muy creativa, con una excelente disposición de los componentes, colores y	La interfaz es creativa y funcional, con una disposición adecuada de los componentes,	La interfaz es funcional, pero carece de creatividad y algunos componentes están	La interfaz carece de creatividad y presenta un diseño confuso o

	tipografía que favorecen la experiencia de usuario.	aunque algunos aspectos visuales pueden mejorarse.	mal organizados visualmente.	desorganizado que dificulta la experiencia de usuario.
Cumplimiento de criterios de usabilidad	La interfaz es altamente usable, accesible y se adapta perfectamente a las necesidades del usuario. Todas las interacciones son intuitivas y claras.	La interfaz es bastante usable, pero algunos elementos podrían optimizarse para mejorar la experiencia del usuario y la accesibilidad.	La interfaz presenta varios problemas de usabilidad que dificultan la interacción del usuario o su accesibilidad.	La interfaz no es usable ni accesible, presentando graves fallos en la interacción con el usuario.
Desarrollo de la lógica de la interfaz	La lógica detrás de la interfaz está completamente desarrollada, con una interacción coherente entre todos los componentes.	La lógica de la interfaz es adecuada, pero algunos componentes o funcionalidades no se integran de manera completamente fluida.	La lógica de la interfaz es básica y presenta algunos fallos o incoherencias en la interacción de los componentes.	La lógica de la interfaz es insuficiente o errónea, lo que provoca fallos importantes en la interacción de los componentes.
Reflexión crítica y capacidad de mejora	Se ha demostrado una reflexión crítica profunda sobre el diseño de su interfaz, proponiendo mejoras innovadoras y realistas.	Se ha reflexionado sobre el diseño de su interfaz, proponiendo mejoras útiles, aunque algo convencionales.	Se han presentado algunas mejoras, pero no hay una reflexión crítica sólida sobre su diseño o el de otros.	No se ha reflexionado de manera adecuada sobre su diseño y no ha propuesto mejoras relevantes.